Greedy Algorithms on Graphs

Shortest paths and minimum spanning trees

AIL7002 · Data Structures and Algorithms (refresher)

IIT Delhi

Today: greedy on graphs

A greedy algorithm builds a solution one irrevocable choice at a time, always taking what looks best *now*.

Two ways to prove such an algorithm optimal:

- Greedy stays ahead: after every step, greedy's partial solution is at least as good as any other.
- Exchange argument: swap pieces of any optimal solution into greedy's without making it worse.

Problem / tool	Proof idea
Shortest paths (Dijkstra)	stays ahead
MST: Kruskal, Prim	exchange (cut property)

Shortest paths: Dijkstra's algorithm

The single-source shortest-path problem

Input: directed graph $G = (V, E)$, start node s, edge lengths $\ell_e \geq 0$.

Length of a path: $\ell(P) = \sum_{e \in P} \ell_e$.

Goal: a shortest s–v path for every $v \in V$.

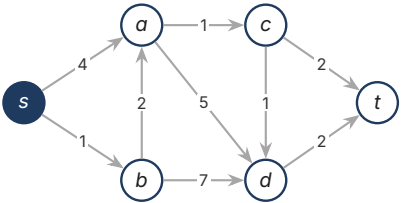
The single-source shortest-path problem

Input: directed graph $G = (V, E)$, start node s, edge lengths $\ell_e \geq 0$.

Length of a path: $\ell(P) = \sum_{e \in P} \ell_e$.

Goal: a shortest s–v path for every $v \in V$.

- Undirected graph? Replace each edge $\{u, v\}$ by (u, v) and (v, u) of the same length.
- We first compute the *distances* $d(v)$; the paths come almost for free.



Our running example.

Dijkstra's algorithm (1959)

Grow a set S of explored nodes whose distance $d(u)$ is known. For every $v \notin S$ consider the best path that runs *inside* S and then takes one edge out:

$$d'(v) = \min_{e=(u,v):\, u \in S} d(u) + \ell_e.$$

Dijkstra's algorithm (1959)

Grow a set S of explored nodes whose distance $d(u)$ is known. For every $v \notin S$ consider the best path that runs *inside* S and then takes one edge out:

$$d'(v) = \min_{e=(u,v):\, u \in S} d(u) + \ell_e.$$

Greedy choice: add the $v \notin S$ with the smallest $d'(v)$, and set $d(v) = d'(v)$.

Dijkstra's algorithm (1959)

Grow a set S of explored nodes whose distance $d(u)$ is known. For every $v \notin S$ consider the best path that runs *inside* S and then takes one edge out:

$$d'(v) = \min_{e=(u,v):\, u \in S} d(u) + \ell_e.$$

Greedy choice: add the $v \notin S$ with the smallest $d'(v)$, and set $d(v) = d'(v)$.

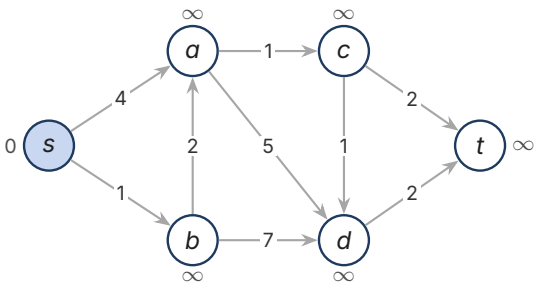
Record the edge (u, v) that achieved the minimum: following these edges backwards from v gives the path P_v.

Dijkstra(G, ℓ)

$S \leftarrow \{s\}$; $d(s) \leftarrow 0$
while $S \neq V$ **do**
  pick $v \notin S$ (with an edge from S)
  with the smallest $d'(v)$
  $S \leftarrow S \cup \{v\}$; $d(v) \leftarrow d'(v)$
end while

Physical picture: a wave spreading from s through pipes reaches nodes in order of distance — exactly Dijkstra's order.

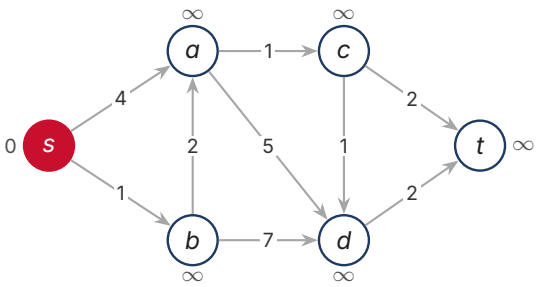
Dijkstra, step by step



v	$d'(v)$	pred
s	0	–
a	∞	–
b	∞	–
c	∞	–
d	∞	–
t	∞	–

Initialise: $d'(s) = 0$, every other key ∞. All nodes sit in the priority queue.

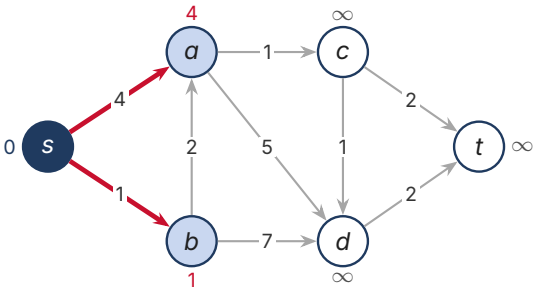
Dijkstra, step by step



v	$d'(v)$	pred	
s	0	–	**extract**
a	∞	–	
b	∞	–	
c	∞	–	
d	∞	–	
t	∞	–	

ExtractMin $\Rightarrow$ s with $d(s) = 0$. Its distance is now final; s joins S.

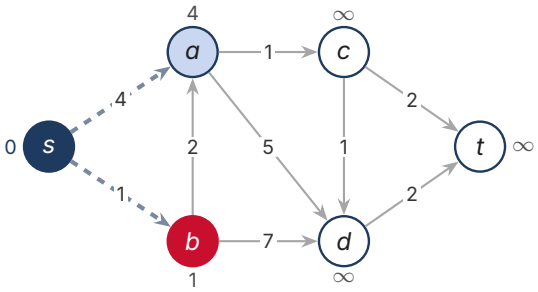
Dijkstra, step by step



v	$d'(v)$	pred	
s	0	–	in S
a	**4**	s	
b	**1**	s	
c	∞	–	
d	∞	–	
t	∞	–	

Relax edges out of s: a: $0 + 4 = 4 < \infty$ **update**; b: $0 + 1 = 1 < \infty$ **update**.

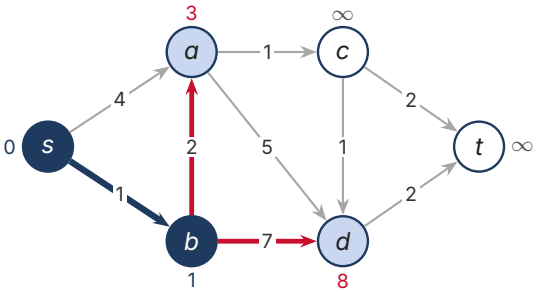
Dijkstra, step by step



v	$d'(v)$	pred	
s	0	–	in S
a	4	s	
b	1	s	**extract**
c	∞	–	
d	∞	–	
t	∞	–	

ExtractMin $\Rightarrow$ b with $d(b) = 1$. Its distance is now final; b joins S.

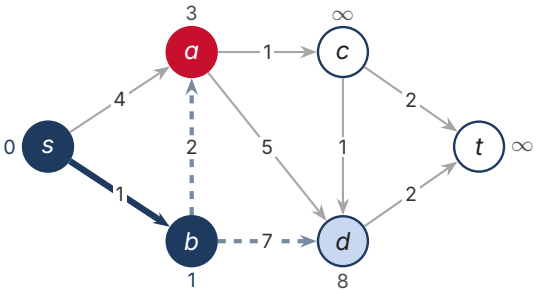
Dijkstra, step by step



v	$d'(v)$	pred	
s	0	–	in S
a	**3**	b	
b	1	s	in S
c	∞	–	
d	**8**	b	
t	∞	–	

Relax edges out of b: a: $1+2=3<4$ **update**; d: $1+7=8<\infty$ **update**.

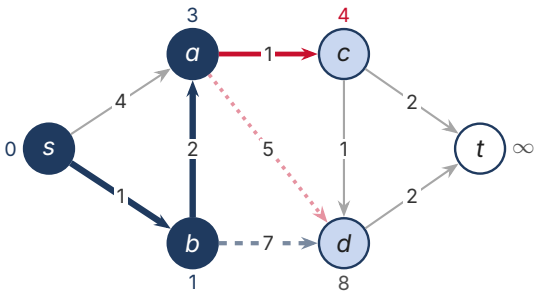
Dijkstra, step by step



v	$d'(v)$	pred	
s	0	–	in S
a	3	b	**extract**
b	1	s	in S
c	∞	–	
d	8	b	
t	∞	–	

ExtractMin $\Rightarrow$ a with $d(a) = 3$. Its distance is now final; a joins S.

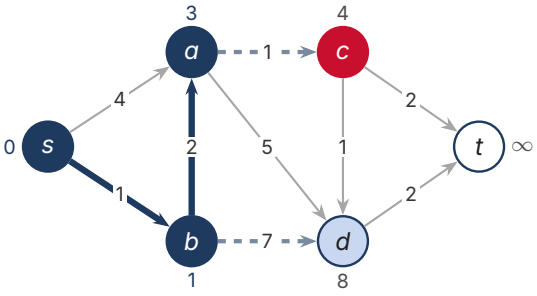
Dijkstra, step by step



v	$d'(v)$	pred	
s	0	–	in S
a	3	b	in S
b	1	s	in S
c	**4**	a	
d	8	b	
t	∞	–	

Relax edges out of a: c: $3+1=4<\infty$ **update**; d: $3+5=8\not<8$ (keep).

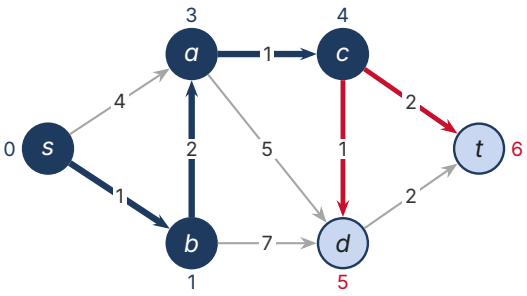
Dijkstra, step by step



v	$d'(v)$	pred	
s	0	–	in S
a	3	b	in S
b	1	s	in S
c	4	a	**extract**
d	8	b	
t	∞	–	

ExtractMin $\Rightarrow$ c with $d(c) = 4$. Its distance is now final; c joins S.

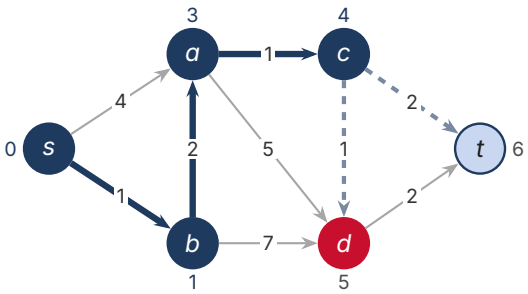
Dijkstra, step by step



v	$d'(v)$	pred	
s	0	–	in S
a	3	b	in S
b	1	s	in S
c	4	a	in S
d	**5**	c	
t	**6**	c	

Relax edges out of c: d: $4 + 1 = 5 < 8$ **update**; t: $4 + 2 = 6 < \infty$ **update**.

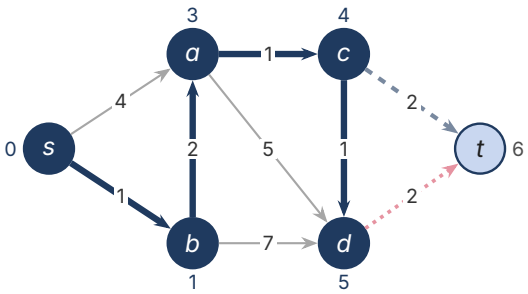
Dijkstra, step by step



v	$d'(v)$	pred	
s	0	–	in S
a	3	b	in S
b	1	s	in S
c	4	a	in S
d	5	c	**extract**
t	6	c	

ExtractMin $\Rightarrow$ d with $d(d) = 5$. Its distance is now final; d joins S.

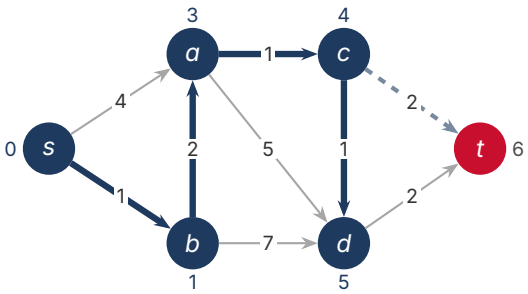
Dijkstra, step by step



v	$d'(v)$	pred	
s	0	–	in S
a	3	b	in S
b	1	s	in S
c	4	a	in S
d	5	c	in S
t	6	c	

Relax edges out of d: t: $5 + 2 = 7 \not< 6$ (keep).

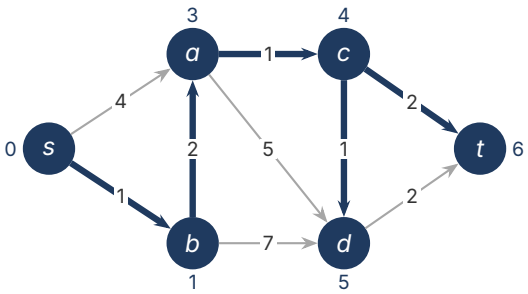
Dijkstra, step by step



v	$d'(v)$	pred	
s	0	–	in S
a	3	b	in S
b	1	s	in S
c	4	a	in S
d	5	c	in S
t	6	c	**extract**

ExtractMin $\Rightarrow$ t with $d(t) = 6$. Its distance is now final; t joins S.

Dijkstra, step by step



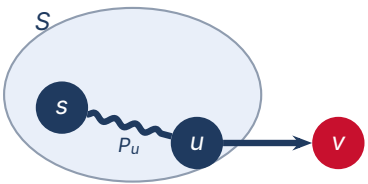
v	$d'(v)$	pred	
s	0	–	in S
a	3	*b*	in S
b	1	*s*	in S
c	4	*a*	in S
d	5	*c*	in S
t	6	*c*	in S

Done. The **pred** edges form a *shortest-path tree*: follow pred back to s to read off P_v.

Why it is correct: greedy stays ahead

Invariant

At every point of the algorithm, for each $u \in S$ the path P_u is a shortest s–u path.

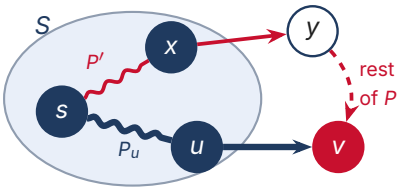


Induction on $|S|$. Suppose v is added via (u, v); take *any* other s–v path P.

Why it is correct: greedy stays ahead

Invariant

At every point of the algorithm, for each $u \in S$ the path P_u is a shortest s–u path.

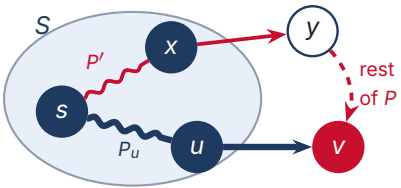


Induction on $|S|$. Suppose v is added via (u, v); take *any* other s–v path P. It must leave S: let y be its first node outside S, $x \in S$ the node before.

Why it is correct: greedy stays ahead

Invariant

At every point of the algorithm, for each $u \in S$ the path P_u is a shortest s–u path.



Induction on $|S|$. Suppose v is added via (u, v); take *any* other s–v path P. It must leave S: let y be its first node outside S, $x \in S$ the node before.

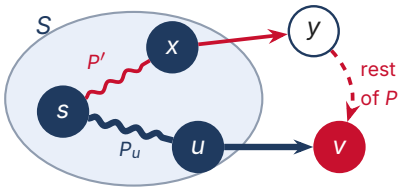
$$
\begin{aligned}
\ell(P) &\geq \ell(P') + \ell_{xy} && \text{rest of } P \text{ has length} \geq 0 \\
&\geq d(x) + \ell_{xy} && \text{induction: } P_x \text{ is shortest} \\
&\geq d'(y) && \text{definition of } d'(y) \\
&\geq d'(v) = \ell(P_v) && \text{greedy chose } v, \text{ not } y
\end{aligned}
$$

P is already too long when it leaves S.

Why it is correct: greedy stays ahead

Invariant

At every point of the algorithm, for each $u \in S$ the path P_u is a shortest s–u path.

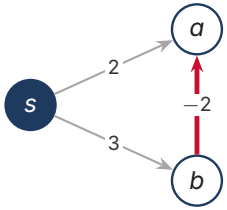


Induction on $|S|$. Suppose v is added via (u, v); take *any* other s–v path P. It must leave S: let y be its first node outside S, $x \in S$ the node before.

$$
\begin{aligned}
\ell(P) &\geq \ell(P') + \ell_{xy} && \text{rest of } P \text{ has length} \geq 0\text{: needs } \ell_e \geq 0 \\
&\geq d(x) + \ell_{xy} && \text{induction: } P_x \text{ is shortest} \\
&\geq d'(y) && \text{definition of } d'(y) \\
&\geq d'(v) = \ell(P_v) && \text{greedy chose } v\text{, not } y
\end{aligned}
$$

P is already too long when it leaves S.

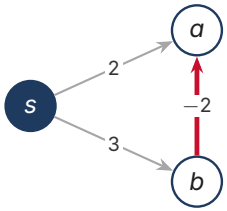
Negative edge lengths



Quiz

What does Dijkstra report as $d(a)$? What is the true distance?

Negative edge lengths

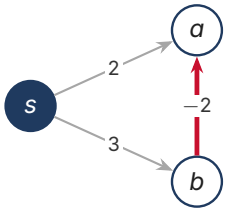


Quiz

What does Dijkstra report as $d(a)$? What is the true distance?

Dijkstra extracts a first with $d(a) = 2$ and never revisits it; but $s \to b \to a$ has length $3 - 2 = 1$.

Negative edge lengths



Quiz

What does Dijkstra report as $d(a)$? What is the true distance?

Dijkstra extracts a first with $d(a) = 2$ and never revisits it; but $s \to b \to a$ has length $3 - 2 = 1$.

The proof breaks at "the rest of P has length ≥ 0". Negative lengths need Bellman–Ford (dynamic programming, later).

Implementation: a priority queue

Keep $V - S$ in a priority queue keyed by $d'(v)$.

- next node: ExtractMin (n times)
- add v: for each (v, w), $d'(w) \leftarrow \min(d'(w),\ d(v) + \ell_{vw})$ by ChangeKey ($\leq m$ times)

Heap: each operation $O(\log n)$
$\Rightarrow$ Dijkstra runs in $O(m \log n)$.

C++: no ChangeKey in `priority_queue`, so push a new entry and skip stale ones.

Added (not in the book) – compiled and tested:

```cpp
// adj[u] = list of (v, len), all len >= 0
vector<ll> dijkstra(const vector<vector<pii>>& adj,
                    int s) {
    vector<ll> d(adj.size(), LLONG_MAX);
    priority_queue<pair<ll,int>, vector<pair<ll,int>>,
                   greater<>> pq;      // min-heap
    d[s] = 0; pq.push({0, s});
    while (!pq.empty()) {
        auto [du, u] = pq.top(); pq.pop(); // ExtractMin
        if (du != d[u]) continue;      // stale entry
        for (auto [v, len] : adj[u])
            if (d[u] + len < d[v]) {  // relax (u,v)
                d[v] = d[u] + len;
                pq.push({d[v], v});   // "ChangeKey"
            }
    }
    return d;
}
```

Minimum spanning trees

The minimum spanning tree problem

Input: connected undirected graph $G = (V, E)$, costs $c_e > 0$.

Goal: $T \subseteq E$ such that (V, T) is connected and $\sum_{e \in T} c_e$ is minimum.

E.g. the cheapest network of links connecting n sites.

The minimum spanning tree problem

Input: connected undirected graph $G = (V, E)$, costs $c_e > 0$.

Goal: $T \subseteq E$ such that (V, T) is connected and $\sum_{e \in T} c_e$ is minimum.

E.g. the cheapest network of links connecting n sites.

The optimum is a tree

If (V, T) had a cycle C, deleting any edge of C keeps it connected (go "the long way" round C) and is cheaper.

The minimum spanning tree problem

Input: connected undirected graph $G = (V, E)$, costs $c_e > 0$.

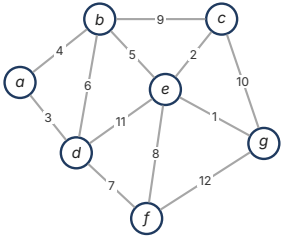
Goal: $T \subseteq E$ such that (V, T) is connected and $\sum_{e \in T} c_e$ is minimum.

E.g. the cheapest network of links connecting n sites.

The optimum is a tree

If (V, T) had a cycle C, deleting any edge of C keeps it connected (go "the long way" round C) and is cheaper.

So: find the cheapest spanning tree — among exponentially many.



Running example: 7 nodes, 12 edges, distinct costs.

Three greedy algorithms — all optimal

- Kruskal: go through edges in *increasing* cost; add e unless it creates a cycle.

Three greedy algorithms — all optimal

- Kruskal: go through edges in *increasing* cost; add e unless it creates a cycle.
- Prim: grow a tree from a root s (like Dijkstra); each round add the node $v \notin S$ with the cheapest *attachment cost* $a(v) = \min_{(u,v):u \in S} c_{uv}$.

Three greedy algorithms — all optimal

- Kruskal: go through edges in *increasing* cost; add e unless it creates a cycle.
- Prim: grow a tree from a root s (like Dijkstra); each round add the node $v \notin S$ with the cheapest *attachment cost* $a(v) = \min_{(u,v):u \in S} c_{uv}$.
- Reverse-Delete: go through edges in *decreasing* cost; delete e unless that disconnects the graph.

Three greedy algorithms — all optimal

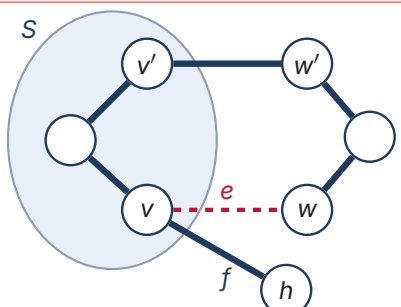
- Kruskal: go through edges in *increasing* cost; add e unless it creates a cycle.
- Prim: grow a tree from a root s (like Dijkstra); each round add the node $v \notin S$ with the cheapest *attachment cost* $a(v) = \min_{(u,v):u \in S} c_{uv}$.
- Reverse-Delete: go through edges in *decreasing* cost; delete e unless that disconnects the graph.

Rare for a problem: almost every natural greedy rule works. Why? Two simple facts — the cut property and the cycle property.

When is it safe to *include* an edge? The cut property

Cut property (distinct costs)

Let $\emptyset \neq S \subsetneq V$ and let $e = (v, w)$ be the cheapest edge with one end in S and the other in $V - S$. Then *every* MST contains e.

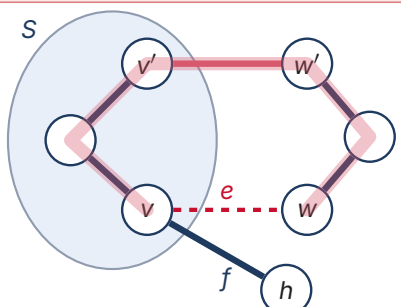


Exchange argument. Take a spanning tree T (thick) with $e \notin T$ (dashed). We build a *cheaper* spanning tree.

When is it safe to *include* an edge? The cut property

Cut property (distinct costs)

Let $\emptyset \neq S \subsetneq V$ and let $e = (v, w)$ be the cheapest edge with one end in S and the other in $V - S$. Then *every* MST contains e.

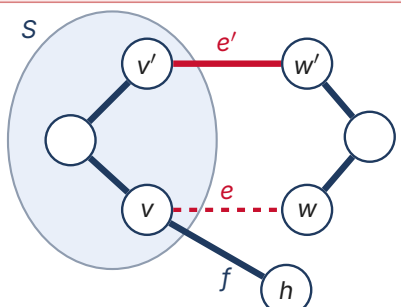


T is a spanning tree, so it has a path P from v to w. P starts in S and ends outside S...

When is it safe to *include* an edge? The cut property

Cut property (distinct costs)

Let $\emptyset \neq S \subsetneq V$ and let $e = (v, w)$ be the cheapest edge with one end in S and the other in $V - S$. Then *every* MST contains e.

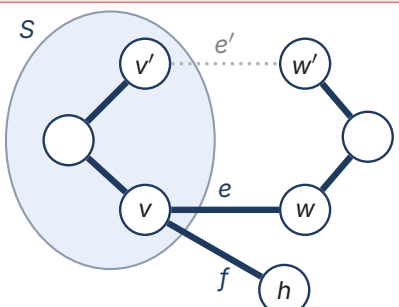


...so it has a first edge $e' = (v', w')$ that crosses from S to $V - S$.

When is it safe to *include* an edge? The cut property

Cut property (distinct costs)

Let $\emptyset \neq S \subsetneq V$ and let $e = (v, w)$ be the cheapest edge with one end in S and the other in $V - S$. Then *every* MST contains e.



Swap: $T' = T - \{e'\} \cup \{e\}$.

- connected: reroute through $v' \rightsquigarrow v \to w \rightsquigarrow w'$;
- acyclic: the only cycle of $T \cup \{e\}$ was $P + e$,

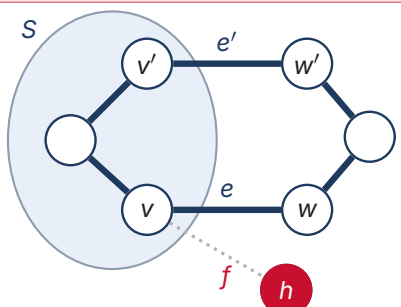
and we cut it.

e is the cheapest crossing edge, so $c_e < c_{e'}$: T' is cheaper than T. $\square$

When is it safe to *include* an edge? The cut property

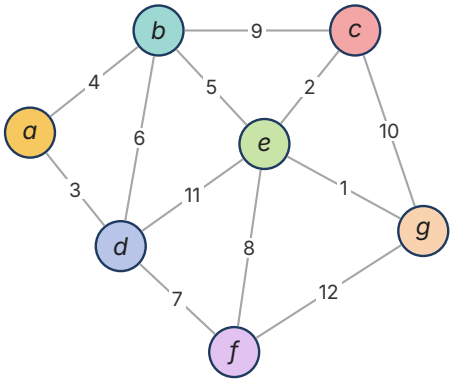
Cut property (distinct costs)

Let $\emptyset \neq S \subsetneq V$ and let $e = (v, w)$ be the cheapest edge with one end in S and the other in $V - S$. Then *every* MST contains e.



Subtle point: we cannot swap e for *any* crossing edge of T. Swapping out f instead would cut h off — not a spanning tree. We need the crossing edge *on the path* P.

Kruskal's algorithm on the example

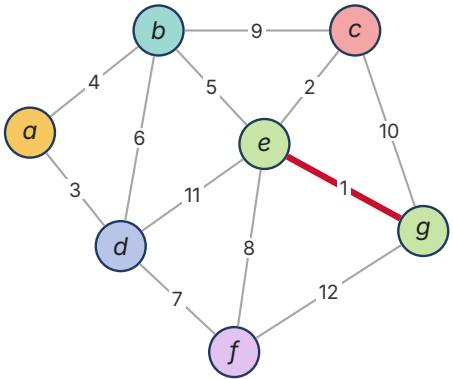


edge	c_e
eg	1
ce	2
ad	3
ab	4
be	5
bd	6

edge	c_e
df	7
ef	8
bc	9
cg	10
de	11
fg	12

Sort the edges by cost. Every node starts in its own component (one colour per component).

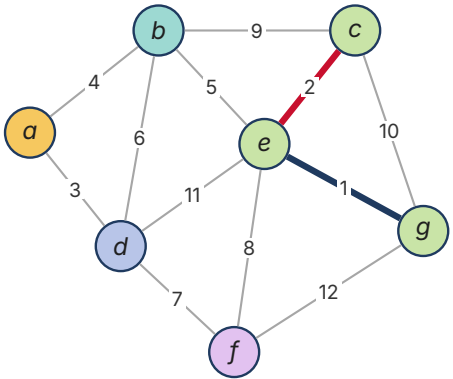
Kruskal's algorithm on the example



edge	c_e		edge	c_e
eg	**1**	✓	*df*	7
ce	2		*ef*	8
ad	3		*bc*	9
ab	4		*cg*	10
be	5		*de*	11
bd	6		*fg*	12

Edge *eg* (cost 1): Find(e) $\neq$ Find(g) $\Rightarrow$ **add** it and Union the two components.

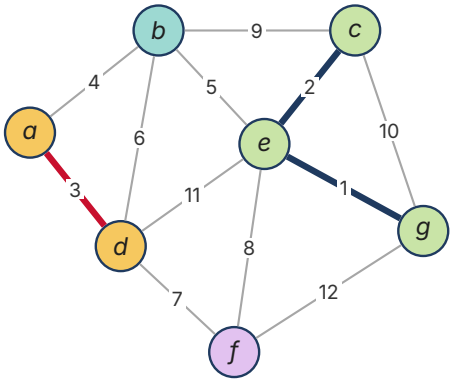
Kruskal's algorithm on the example



edge	c_e		edge	c_e
eg	1	✓	*df*	7
ce	**2**	✓	*ef*	8
ad	3		*bc*	9
ab	4		*cg*	10
be	5		*de*	11
bd	6		*fg*	12

Edge *ce* (cost 2): Find(c) $\neq$ Find(e) $\Rightarrow$ **add** it and Union the two components.

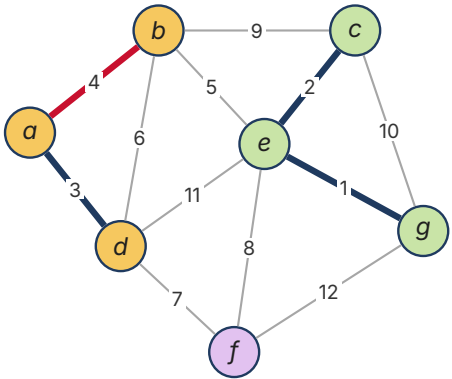
Kruskal's algorithm on the example



edge	c_e		edge	c_e
eg	1	✓	*df*	7
ce	2	✓	*ef*	8
ad	**3**	✓	*bc*	9
ab	4		*cg*	10
be	5		*de*	11
bd	6		*fg*	12

Edge *ad* (cost 3): Find(a) $\neq$ Find(d) $\Rightarrow$ **add** it and Union the two components.

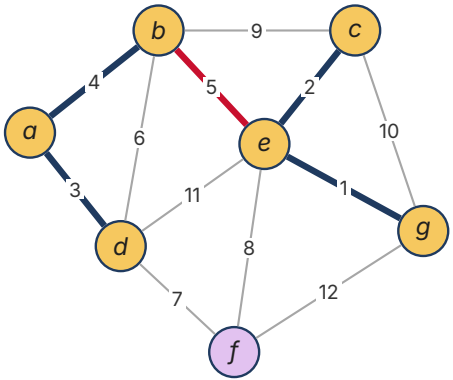
Kruskal's algorithm on the example



edge	c_e		edge	c_e
eg	1	✓	*df*	7
ce	2	✓	*ef*	8
ad	3	✓	*bc*	9
ab	**4**	✓	*cg*	10
be	5		*de*	11
bd	6		*fg*	12

Edge *ab* (cost 4): Find(a) $\neq$ Find(b) $\Rightarrow$ **add** it and Union the two components.

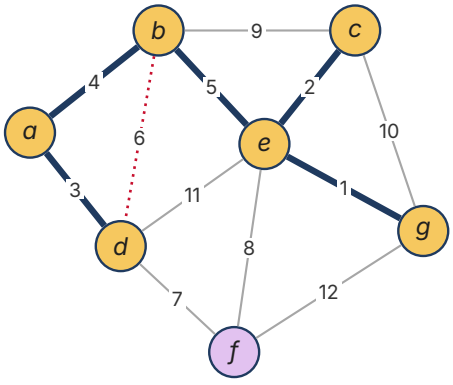
Kruskal's algorithm on the example



edge	c_e		edge	c_e
eg	1	✓	*df*	7
ce	2	✓	*ef*	8
ad	3	✓	*bc*	9
ab	4	✓	*cg*	10
be	**5**	✓	*de*	11
bd	6		*fg*	12

Edge *be* (cost 5): Find(b) $\neq$ Find(e) $\Rightarrow$ **add** it and Union the two components.

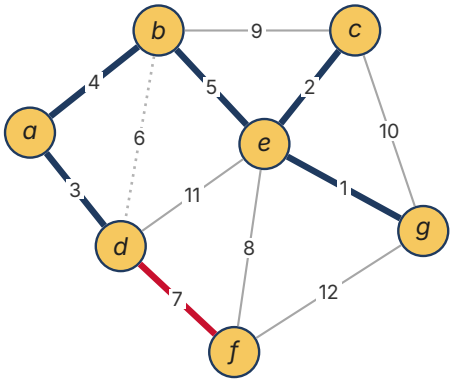
Kruskal's algorithm on the example



edge	c_e		edge	c_e
eg	1	✓	*df*	7
ce	2	✓	*ef*	8
ad	3	✓	*bc*	9
ab	4	✓	*cg*	10
be	5	✓	*de*	11
bd	**6**	×	*fg*	12

Edge *bd* (cost 6): Find(b) = Find(d) ⇒ it would close a cycle: **discard**.

Kruskal's algorithm on the example

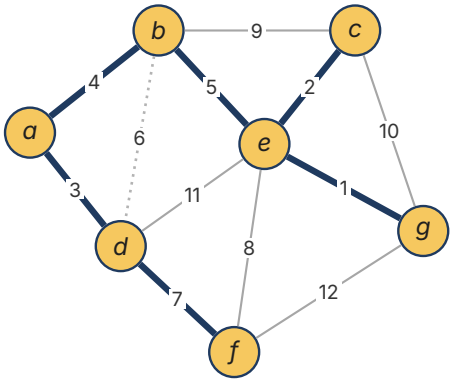


edge	c_e	
eg	1	✓
ce	2	✓
ad	3	✓
ab	4	✓
be	5	✓
bd	6	×

edge	c_e	
df	**7**	✓
ef	8	
bc	9	
cg	10	
de	11	
fg	12	

Edge *df* (cost 7): Find(d) $\neq$ Find(f) $\Rightarrow$ **add** it and Union the two components.

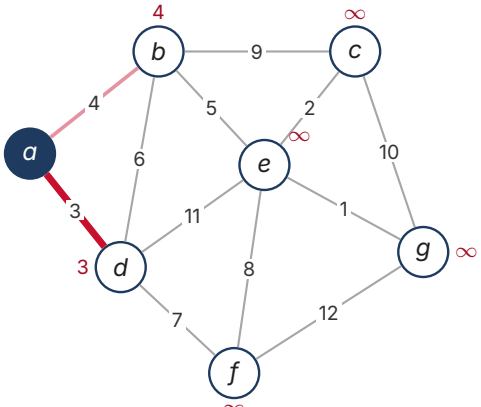
Kruskal's algorithm on the example



edge	c_e		edge	c_e	
eg	1	✓	df	7	✓
ce	2	✓	ef	8	
ad	3	✓	bc	9	
ab	4	✓	cg	10	
be	5	✓	de	11	
bd	6	×	fg	12	

$n - 1 = 6$ edges accepted $\Rightarrow$ spanning tree; we can stop early. Total cost 22.

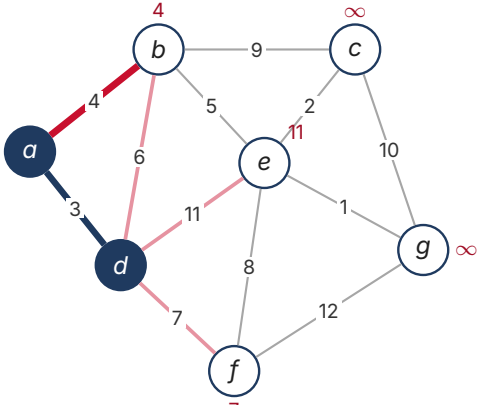
Prim's algorithm on the same graph



v	$a(v)$	via
a		in S
b	4	a
c	∞	–
d	**3**	a
e	∞	–
f	∞	–
g	∞	–

Start from $S = \{a\}$. Edges crossing the cut $(S, V - S)$ are highlighted; the cheapest one is thick.

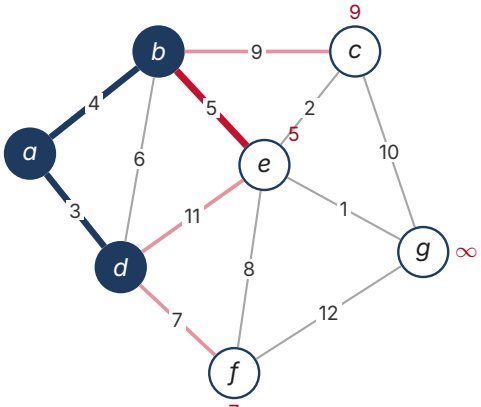
Prim's algorithm on the same graph



v	$a(v)$	via
a		in S
b	**4**	a
c	∞	–
d		in S
e	11	d
f	7	d
g	∞	–

Added the cheapest crossing edge ad (cost 3): d joins S; update $a(\cdot)$ for d's neighbours. Next cheapest crossing edge (thick): ab.

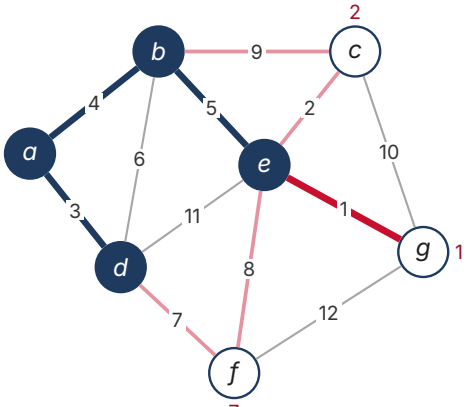
Prim's algorithm on the same graph



v	$a(v)$	via
a		in S
b		in S
c	9	b
d		in S
e	**5**	b
f	7	d
g	∞	–

Added the cheapest crossing edge ab (cost 4): b joins S; update $a(\cdot)$ for b's neighbours. Next cheapest crossing edge (thick): be.

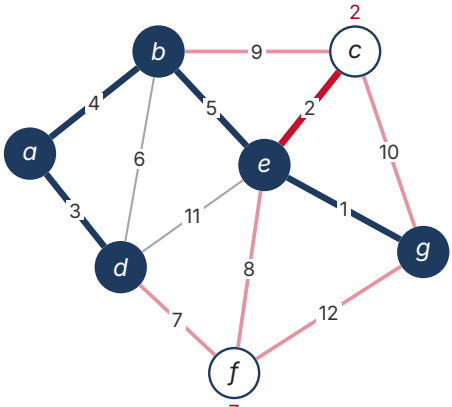
Prim's algorithm on the same graph



v	$a(v)$	via
a		in S
b		in S
c	2	e
d		in S
e		in S
f	7	d
g	**1**	e

Added the cheapest crossing edge be (cost 5): e joins S; update $a(\cdot)$ for e's neighbours. Next cheapest crossing edge (thick): eg.

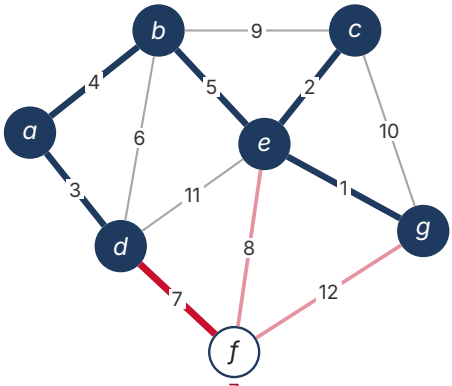
Prim's algorithm on the same graph



v	$a(v)$	via
a		in S
b		in S
c	**2**	e
d		in S
e		in S
f	7	d
g		in S

Added the cheapest crossing edge eg (cost 1): g joins S; update $a(\cdot)$ for g's neighbours. Next cheapest crossing edge (thick): ce.

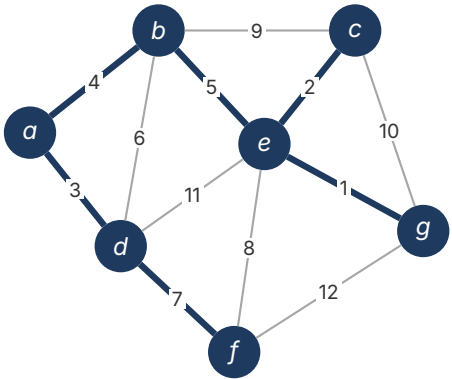
Prim's algorithm on the same graph



v	$a(v)$	via
a		in S
b		in S
c		in S
d		in S
e		in S
f	**7**	d
g		in S

Added the cheapest crossing edge ce (cost 2): c joins S; update $a(\cdot)$ for c's neighbours. Next cheapest crossing edge (thick): df.

Prim's algorithm on the same graph



v	$a(v)$	via
a		in S
b		in S
c		in S
d		in S
e		in S
f		in S
g		in S

Attach f via df. Done: the same tree Kruskal found, cost 22.

Both are optimal — and the cycle property

Kruskal. When $e = (v, w)$ is added, let S = nodes reachable from v in the current forest. $w \notin S$ (no cycle), and no earlier edge crosses $(S, V - S)$ (it would have been added). So e is the cheapest crossing edge $\Rightarrow$ in every MST. Output has no cycle, and is connected since G is.

Prim. Each added edge is by definition the cheapest edge leaving S. $\square$

Both are optimal — and the cycle property

Kruskal. When $e = (v, w)$ is added, let S = nodes reachable from v in the current forest. $w \notin S$ (no cycle), and no earlier edge crosses $(S, V - S)$ (it would have been added). So e is the cheapest crossing edge $\Rightarrow$ in every MST. Output has no cycle, and is connected since G is.

Prim. Each added edge is by definition the cheapest edge leaving S. $\square$

Cycle property

If e is the most expensive edge on some cycle C, then e is in *no* MST.

Same exchange idea (swap e out for a cheaper edge of C). Justifies **Reverse-Delete**.

Both are optimal — and the cycle property

Kruskal. When $e = (v, w)$ is added, let S = nodes reachable from v in the current forest. $w \notin S$ (no cycle), and no earlier edge crosses $(S, V - S)$ (it would have been added). So e is the cheapest crossing edge $\Rightarrow$ in every MST. Output has no cycle, and is connected since G is.

Prim. Each added edge is by definition the cheapest edge leaving S. $\square$

Cycle property

If e is the most expensive edge on some cycle C, then e is in *no* MST.

Same exchange idea (swap e out for a cheaper edge of C). Justifies **Reverse-Delete**.

Ties? Perturb the costs by tiny distinct amounts: comparisons between different costs are unchanged, so an MST for the perturbed costs is an MST for the originals.

Running times: Prim $\approx$ Dijkstra, Kruskal needs a new tool

Prim

Same code as Dijkstra with key $a(v) = \min_{u \in S} c_{uv}$ instead of $d(u) + \ell_{uv}$: n ExtractMin, $\leq m$ ChangeKey $\Rightarrow O(m \log n)$ with a heap.

Kruskal

Sorting: $O(m \log m) = O(m \log n)$.
Then for each edge: *are u and v already connected?*
BFS/DFS each time: $O(n)$ per edge
$\Rightarrow O(mn)$. Too slow.

We need to maintain components of a graph that only *gains* edges $\Rightarrow$ Union-Find.